T1 纪念品分组（souvenir）

每件纪念品都必须放入一个袋子，每个袋子最多两件，价格和不超过 w。要求的是最少袋数。

先把小组内的选择写清楚

当 $n \le 15$ 时，可以直接枚举怎样把纪念品两两配对。用一个二进制数 mask 表示还没有装入袋子的纪念品：第 i 位为 1 表示第 i 件尚未处理。

函数 solve(mask) 返回处理完这些纪念品所需的最少袋数。为了避免同一种分组被不同顺序重复枚举，先取 mask 中编号最小的纪念品 i，规定这一步必须处理掉它。此时只有两类选择：

1. i 单独装袋，余下状态为 mask 去掉第 i 位；
2. 再选一件纪念品 j，若 $a_i + a_j \le w$，让二者同袋，余下状态同时去掉两位。

每个完整分组都会在自己的第一步唯一地确定“与最小未处理物品同袋的对象”，因此枚举不会遗漏。相同 mask 的后续问题完全一样，可以记忆化。状态数为 2^n，每个状态最多再枚举 n 个配对对象，时间为 $O(n2^n)$，空间为 $O(2^n)$。

例如取 $a = (1, 2, 3, 4)$、$w = 5$。从剩余集合 $\{0, 1, 2, 3\}$ 出发，若把 0 与 1 配对，就到达 $\{2, 3\}$；先从 $\{1, 2, 3\}$ 单独处理 1，也会再次到达 $\{2, 3\}$。这张局部状态图展示的是“已经处理过的物品集合”相同，所以后续选择相同。

```
{0,1,2,3}
  0 = 单独处理 0；剩余 {1,2,3}
  1 = 令 0 与 1 配对；剩余 {2,3}
  2 = 令 0 与 2 配对；剩余 {1,3}
  3 = 令 0 与 3 配对；剩余 {1,2}

{1,2,3}
  0 = 单独处理 1；剩余 {2,3}   <- 与上一行编号 1 合流
  1 = 令 1 与 2 配对；剩余 {3}
  2 = 令 1 与 3 配对；条件 2+4<=5 不成立，不是合法转移

合法叶子：mask=0，返回 0。
非法转移：两件价格之和超过 w。
箭头方向：从剩余集合走向处理后的剩余集合。
```

这套状态能取得测试点 $1 \sim 8$ 的保证分。

```cpp
#include<bits/stdc++.h>
using namespace std;

const int N = 15;
int n,w,a[N],dp[1 << N];

int solve(int mask){
    if(mask == 0) return 0;
    if(dp[mask] != -1) return dp[mask];
    int i = 0;
    while(((mask >> i) & 1) == 0) i++;
    int rest = mask ^ (1 << i);
    int ans = 1 + solve(rest);
    for(int j = i + 1; j < n; j++){
        if(((mask >> j) & 1) && a[i] + a[j] <= w){
            ans = min(ans,1 + solve(rest ^ (1 << j)));
        }
    }
    dp[mask] = ans;
    return ans;
}

int main(){
```

```
    freopen("souvenir.in","r",stdin);
    freopen("souvenir.out","w",stdout);
    cin >> w >> n;
    for(int i = 0; i < n; i++) cin >> a[i];
    memset(dp,-1,sizeof(dp));
    cout << solve((1 << n) - 1) << '\n';
    return 0;
}
```

最小价格和最大价格能否同袋

把价格从小到大排序，设为

$$a_1 \le a_2 \le \cdots \le a_n.$$

当前最轻的是 a_l，最重的是 a_r。

如果 $a_l + a_r > w$，最重的 a_r 与任何剩余纪念品相加都不可能不超过 w，所以它只能单独成袋，直接删除 a_r。

如果 $a_l + a_r \le w$，可以断言存在一种最优方案让二者同袋。假设某个最优方案中，a_r 与 a_y 同袋，a_l 与 a_x 同袋。若交换成 (a_l, a_r) 与 (a_x, a_y)，第一组合法。原来有 $a_r + a_y \le w$，又因为 $a_x \le a_r$，所以 $a_x + a_y \le a_r + a_y \le w$，第二组也合法，袋数不变。若其中一件原本单独成袋，把两者配在一起并把原来的搭档改成单独的袋子，同样不增加袋数。因此把最轻和最重配对不会损失最优性。

于是每一步只有两种处理：

- 两端和超过 w，最重者单独一袋；
- 否则二者放同一袋。

排序需要 $O(n \log n)$，双指针扫描为 $O(n)$，空间为 $O(n)$。

```
#include<bits/stdc++.h>
using namespace std;

const int N = 30000 + 10;
int n,w,a[N];

int main(){
    freopen("souvenir.in","r",stdin);
    freopen("souvenir.out","w",stdout);
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin >> w >> n;
    for(int i = 1; i <= n; i++) cin >> a[i];
    sort(a + 1,a + n + 1);
    int l = 1,r = n,ans = 0;
    while(l <= r){
        if(l < r && a[l] + a[r] <= w){
            l++;
            r--;
        }else{
            r--;
        }
        ans++;
    }
    cout << ans << '\n';
    return 0;
}
```

看到“每袋至多两件、要求袋数和不超过 w”时，可以先检查最大项能否与任何项共存；若能与最小项共存，就用交换论证把最优方案调整成固定配对。

T2 安全密码（password）

每个位置允许一个数字区间。要判断是否存在一个长度为 m 的数字串，它不是 s 的子序列。

直接枚举允许的密码

当 $m \le 3$ 时，每一位至多只有 10 种选择，因此至多尝试 10^m 个候选密码。对每个候选，从左到右扫描 s，尽可能按顺序匹配它的每一位；若最终有位没匹配完，就说明该候选不是子序列。

这是一份基于题意定义的完整程序，只用于小规模正确性检查。最坏时间与候选数和字符串长度有关，不能直接用于一般的 $m \le 10$。

```
#include<bits/stdc++.h>
using namespace std;

string s;
int m;

bool isSubsequence(const string &p){
    int j = 0;
    for(int i = 0; i < (int)s.size() && j < m; i++){
        if(s[i] == p[j]) j++;
    }
    return j == m;
}

bool dfs(int pos,string &l,string &r,string &p){
    if(pos == m) return !isSubsequence(p);
    for(char c = l[pos]; c <= r[pos]; c++){
        p.push_back(c);
        if(dfs(pos + 1,l,r,p)) return true;
        p.pop_back();
    }
    return false;
}

int main(){
    freopen("password.in","r",stdin);
    freopen("password.out","w",stdout);
    int t;
    cin >> t;
    while(t--){
        string l,r,p;
        cin >> s >> m >> l >> r;
        cout << (dfs(0,l,r,p) ? "YES" : "NO") << '\n';
    }
    return 0;
}
```

预处理每个数字的下一次出现位置

单独判断一个候选时，反复在 s 中 find 会重复扫描同一段字符。若已经匹配到下标 p，下一步真正需要的只是：从 $p+1$ 开始，每个数字第一次出现在哪里。

定义 nxt[d][i] 为下标 i 及其后第一个等于数字 d 的位置；如果不存在，值为 $|s|$。从右向左计算：

$$nxt[d][i] = \begin{cases} i, & s_i = d, \\ nxt[d][i+1], & s_i \neq d. \end{cases}$$

哨兵 nxt[d][n]=n 表示“不存在”。这样每次查询只需要一次数组访问。

为什么只保留最晚的位置

设当前已经匹配了前缀，下一步允许的数字组成区间 $[l_i, r_i]$。对区间内每个数字 d，令 $p_d = nxt[d][\text{cur}+1]$。

- 若某个 $p_d = |s|$，就可以直接选数字 d，并任意补完后缀。所得密码不可能是 s 的子序列，答案为 YES。
- 否则所有允许数字都能继续匹配。对于后续决策，当前位置越靠右，留给后面的字符越少。如果从位置 x 出发存在一种补法使密码无法匹配，那么从更靠右的 $y > x$ 出发，这种补法更不可能变得可匹配。因此只需研究所选位置最靠右的分支，即令 $\text{far} = \max p_d$。

如果 m 位都能按上述过程继续，说明无论每一步选择哪个允许数字，后面的密码都仍能被匹配；因此不存在非子序列，答案为 NO。

建表为 $O(10|s|)$，判断一位至多检查 10 个数字。单个测试用例的时间为 $O(10|s| + 10m)$，空间为 $O(10|s|)$。所有字符串总长不超过 3×10^5，该实现覆盖全部数据。

```cpp
#include<bits/stdc++.h>
using namespace std;

const int N = 300000 + 5;
int nxt[10][N];

int main(){
    freopen("password.in","r",stdin);
    freopen("password.out","w",stdout);
    ios::sync_with_stdio(false);
    cin.tie(0);
    int t;
    cin >> t;
    while(t--){
        string s,l,r;
        int m;
        cin >> s;
        cin >> m;
        cin >> l >> r;
        int n = (int)s.size();
        for(int d = 0; d < 10; d++) nxt[d][n] = n;
        for(int i = n - 1; i >= 0; i--){
            for(int d = 0; d < 10; d++) nxt[d][i] = nxt[d][i + 1];
            nxt[s[i] - '0'][i] = i;
        }
        int cur = -1;
        bool found = false;
        for(int i = 0; i < m && !found; i++){
            int far = -1;
            for(int d = l[i] - '0'; d <= r[i] - '0'; d++){
                int p = nxt[d][cur + 1];
                if(p == n){
                    found = true;
                    break;
                }
                far = max(far,p);
            }
            cur = far;
        }
        cout << (found ? "YES" : "NO") << '\n';
    }
    return 0;
}
```

这类“要证明某种选择一定无法完成匹配”的题，若每个字符只有一个后继位置，应优先预处理“下一次出现位置”。本题保留的是最靠右的合法后继，因为它为未来留下的可用后缀最少。

T3 数组升格（growmax）

一次操作只能把左侧元素增加 1，而且增加前必须不大于右邻居。至多操作 k 次，最大化序列中的某个元素。

小规模直接搜索

当 $n \le 5, k \le 6$ 时，每一步至多枚举 $n-1$ 个可以增加的位置。搜索状态只需要当前数组和已经使用的操作次数；到达任意深度或无法操作时，都可以用当前最大值更新答案。

每个合法操作序列都会沿搜索树的一个分支执行，因此不会漏解。最坏时间上界为 $O((n-1)^k n)$，递归深度为 k，空间为 $O(n+k)$。这套做法保证测试点 $1 \sim 4$。

```
#include<bits/stdc++.h>
using namespace std;

int n,k;
int a[10];

int dfs(int used){
    int best = 0;
    for(int i = 1; i <= n; i++) best = max(best,a[i]);
    if(used == k) return best;
    for(int i = 1; i < n; i++){
        if(a[i] <= a[i + 1]){
            a[i]++;
            best = max(best,dfs(used + 1));
            a[i]--;
        }
    }
    return best;
}

int main(){
    freopen("growmax.in","r",stdin);
    freopen("growmax.out","w",stdout);
    int t;
    cin >> t;
    while(t--){
        cin >> n >> k;
        for(int i = 1; i <= n; i++) cin >> a[i];
        cout << dfs(0) << '\n';
    }
    return 0;
}
```

两个元素与递减序列

当 $n = 2$ 时，所有操作都作用在 a_1 上。若 $a_1 > a_2$，一次也不能操作，答案为 a_1。

否则，在第 x 次操作前，左侧值为 $a_1 + x - 1$。它不超过 a_2 当且仅当

$$x \le a_2 - a_1 + 1.$$

所以最多能增加 $\min(k, a_2 - a_1 + 1)$ 次，答案为

$$\max\left(a_2,\ a_1 + \min(k, a_2 - a_1 + 1)\right).$$

当数列严格递减时，$a_1 > a_2 > \cdots > a_n$，没有任何位置满足 $a_i \le a_{i+1}$，答案就是 a_1。

下面的程序同时处理这两种结构，保证测试点 $5 \sim 12$。两分支都只扫描或读取数组，时间 $O(n)$，空间 $O(n)$。

```cpp
#include<bits/stdc++.h>
using namespace std;

int main(){
    freopen("growmax.in","r",stdin);
    freopen("growmax.out","w",stdout);
    int t;
    cin >> t;
    while(t--){
        int n,k;
        cin >> n >> k;
        vector<int> a(n + 1);
        for(int i = 1; i <= n; i++) cin >> a[i];
        long long ans;
        if(n == 2){
            if(a[1] > a[2]){
                ans = a[1];
            }else{
                long long limit = a[2] - a[1] + 1;
                long long gain = min((long long)k,limit);
                ans = max((long long)a[2],(long long)a[1] + gain);
            }
        }else{
            ans = a[1];
        }
        cout << ans << '\n';
    }
    return 0;
}
```

固定要升到的位置和目标值

一般情形中，若最终最大值是 H，它一定出现在某个位置 l。只枚举“在哪个位置办到”还不够，因为即使决定把 a_l 升到 H，还需要处理它右侧的连锁限制。

假设从左向右构造一条升格链：a_l 要升到 H，则它每次增加前都必须不大于 a_{l+1}。当 a_l 变成 $H-1$ 时，下一步若还要增加，a_{l+1} 至少需要是 $H-1$。同理，为了让 a_{l+1} 达到 $H-1$，再右侧需要达到 $H-2$，依此类推。

因此对固定起点 l，各位置的至少目标依次是

$$H,\ H-1,\ H-2,\ldots$$

扫描到某个位置 i 时，若原来的 a_i 已经不小于当前目标 need，这条链就能在这里停止，无需继续支付右侧成本。否则需要增加

$$\max(0, need - a_i) = need - a_i$$

次。再令 need--，检查下一个位置。如果走到序列末尾仍有目标大于 a_n，就不可能把 a_l 升到 H。

所有这些增加都是必要的：要让左侧达到某个值，右侧必须先达到比它小一的值。反过来，从链的右端向左依次增加，每次操作时右邻居已经满足条件，所以支付的总缺口可以真正完成升格。于是检查一个候选 (l, H) 的最少代价就是上述缺口之和。

对所有 l 执行一次检查需要 $O(n^2)$。若存在某个起点可用不超过 k 次操作升到 H，也能升到任何更小的值；反之，若 H 做不到，更大的值也做不到。因此答案具有二分性。

答案至少为原数组最大值，至多增加 k 次，所以二分范围可取 [max(a), max(a)+k]。总时间为 $O(n^2 \log(\max a + k))$，空间为 $O(n)$。在一个输入文件的所有组中，

$$\sum n_i^2 \le \left(\sum n_i\right)^2 \le 10^6,$$

因此还乘上约 27 次二分检查也在范围内。

```cpp
#include<bits/stdc++.h>
using namespace std;

const int N = 1000 + 10;
int n;
long long k,a[N];

bool possible(long long target){
    for(int l = 1; l <= n; l++){
        long long need = target;
        long long cost = 0;
        bool ok = true;
        for(int i = l; i <= n; i++){
            if(a[i] >= need) break;
            if(i == n){
                ok = false;
                break;
            }
            cost += need - a[i];
            if(cost > k) break;
            need--;
        }
        if(ok && cost <= k) return true;
    }
    return false;
}

int main(){
    freopen("growmax.in","r",stdin);
    freopen("growmax.out","w",stdout);
    ios::sync_with_stdio(false);
    cin.tie(0);
    int t;
    cin >> t;
    while(t--){
        cin >> n >> k;
        long long hi = 0;
        for(int i = 1; i <= n; i++){
            cin >> a[i];
            hi = max(hi,a[i]);
        }
        long long low = hi;
        hi += k + 1;
        while(low + 1 < hi){
            long long mid = (low + hi) / 2;
            if(possible(mid)) low = mid;
            else hi = mid;
        }
        cout << low << '\n';
    }
    return 0;
}
```

当一次局部操作受右邻居限制时，可以沿一条方向展开“目标值递减”的连锁成本。先判断某个目标是否可行，再把单点最优问题变成单调的二分答案。

T4 广阔图（widegraph）

树上的距离是一个连续、分层的结构。对每个阈值 k，G_k 只保留树上距离至少为 k 的点对。

显式建出每个图

当 $n \le 10$ 时，可以先从每个顶点出发做一次 DFS，求出任意两点的树上距离。然后对每个 k，初始化并查集，枚举所有点对；若两点距离至少为 k，就合并它们，最后统计集合个数。

距离表需要 $O(n^2)$ 时间与空间；对 n 个阈值分别枚举 $O(n^2)$ 个点对，总时间为 $O(n^3)$，空间为 $O(n^2)$。它保证测试点 $1 \sim 4$。

```
#include<bits/stdc++.h>
using namespace std;

const int N = 12;
int n,d[N][N],fa[N];
vector<int> e[N];

void dfs(int s,int u,int p,int dep){
    d[s][u] = dep;
    for(int i = 0; i < (int)e[u].size(); i++){
        int v = e[u][i];
        if(v != p) dfs(s,v,u,dep + 1);
    }
}

int find(int x){
    if(fa[x] == x) return x;
    fa[x] = find(fa[x]);
    return fa[x];
}

int main(){
    freopen("widegraph.in","r",stdin);
    freopen("widegraph.out","w",stdout);
    cin >> n;
    for(int i = 1; i < n; i++){
        int u,v;
        cin >> u >> v;
        e[u].push_back(v);
        e[v].push_back(u);
    }
    for(int s = 1; s <= n; s++) dfs(s,s,0,0);
    for(int k = 1; k <= n; k++){
        for(int i = 1; i <= n; i++) fa[i] = i;
        for(int u = 1; u <= n; u++){
            for(int v = u + 1; v <= n; v++){
                if(d[u][v] >= k){
                    int a = find(u),b = find(v);
                    if(a != b) fa[a] = b;
                }
            }
        }
        int ans = 0;
        for(int i = 1; i <= n; i++){
            if(find(i) == i) ans++;
        }
        cout << ans << (k == n ? '\n' : ' ');
    }
    return 0;
}
```

链与菊花

若树是一条长度为 $n-1$ 的链，从一个端点沿链走到另一端时，设当前顶点距左端点为 x，则它距右端点为 $n-1-x$，偏心距为

$$e(x) = \max(x, n-1-x).$$

对所有顶点求出 $e(x)$ 后，按下一节的一般计数方法即可在 $O(n \log n)$ 时间内求出全部答案。

若树是菊花图，中心到每个顶点的偏心距为 1，每个叶子到最远叶子的距离为 2。因此：

- $k = 1$：所有边都进入 G_1，答案为 1；
- $k = 2$：只有中心被孤立，答案为 2；
- $k \geq 3$：没有边，答案为 n。

下面的程序根据结构选择链公式或菊花公式，保证测试点 $5 \sim 12$。

```
#include<bits/stdc++.h>
using namespace std;

const int N = 100000 + 10;
int n,deg[N],dist[N];
vector<int> e[N];

void bfs(int s){
    for(int i = 1; i <= n; i++) dist[i] = -1;
    queue<int> q;
    dist[s] = 0;
    q.push(s);
    while(!q.empty()){
        int u = q.front();
        q.pop();
        for(int i = 0; i < (int)e[u].size(); i++){
            int v = e[u][i];
            if(dist[v] == -1){
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
}

int main(){
    freopen("widegraph.in","r",stdin);
    freopen("widegraph.out","w",stdout);
    cin >> n;
    for(int i = 1; i < n; i++){
        int u,v;
        cin >> u >> v;
        e[u].push_back(v);
        e[v].push_back(u);
        deg[u]++;
        deg[v]++;
    }
    bool path = true;
    int center = 0;
    for(int i = 1; i <= n; i++){
        if(deg[i] > 2) path = false;
        if(deg[i] == n - 1) center = i;
    }
    if(path){
        int s = 1;
        for(int i = 1; i <= n; i++){
            if(deg[i] == 1) s = i;
        }
        bfs(s);
        vector<int> ecc(n + 1);
        for(int i = 1; i <= n; i++){
```

```
            ecc[i] = max(dist[i],n - 1 - dist[i]);
        }
        sort(ecc.begin() + 1,ecc.end());
        for(int k = 1; k <= n; k++){
            int ans;
            if(k >= n){
                ans = n;
            }else{
                int isolated = (int)(lower_bound(ecc.begin() + 1,ecc.end(),k) - ecc.begin() - 1);
                ans = 1 + isolated;
            }
            cout << ans << (k == n ? '\n' : ' ');
        }
    }else if(center){
        for(int k = 1; k <= n; k++){
            int ans;
            if(k == 1){
                ans = 1;
            }else if(k == 2){
                ans = 2;
            }else{
                ans = n;
            }
            cout << ans << (k == n ? '\n' : ' ');
        }
    }else{
        for(int k = 1; k <= n; k++) cout << 0 << (k == n ? '\n' : ' ');
    }
    return 0;
}
```

用直径端点概括每个点的最远距离

树的直径是任意两点距离的最大值。先任取一点做 BFS，得到最远点 s；再从 s 做 BFS，得到最远点 t。可以证明 s, t 是一对直径端点。

对任意顶点 v，定义它的偏心距

$$e(v) = \max_u d(u, v).$$

对于直径端点 s, t，有

$$e(v) = \max(d(v, s), d(v, t)).$$

直观上，最远点若从 v 出发分叉到直径之外，把分叉点记为 z；分叉长度受直径限制，无法比走向 s 或 t 更远。形式化地，设 z 在 s, t 路径上的位置为坐标 z，v 到 z 的分叉长度为 h，该分支到某点的额外长度为 g。若 g 超过两个直径端点给出距离中的较大者，则 $g + z$ 或 $g + |s - t| - z$ 会超过直径长度，产生矛盾。因此只需检查两个直径端点。

由这个公式还能把连通块分清楚。设直径为 D。

- 当 $k \le D$ 时，两个直径端点距离为 $D \ge k$，所以它们直接相连，至少有一个主连通块。
- 若 $e(v) \ge k$，由公式可知 v 与 s 或 t 的距离至少为 k，因而 v 与主连通块相连。
- 若 $e(v) < k$，它与任何顶点的距离都小于 k，在 G_k 中没有边，自身就是一个连通块。

所以

$$\text{ans}_k = \begin{cases} 1 + \#\{v : e(v) < k\}, & k \le D, \\ n, & k > D. \end{cases}$$

三次 BFS 得到 s, t 与两个距离数组；计算全部偏心距后排序，再用 lower_bound 统计每个 k 之前有多少个偏心率小于 k。时间为 $O(n \log n)$，空间为 $O(n)$。

```cpp
#include<bits/stdc++.h>
using namespace std;

const int N = 100000 + 10;
int n,dist1[N],dists[N],distt[N],ecc[N];
vector<int> e[N];

void bfs(int s,int dist[]){
    for(int i = 1; i <= n; i++) dist[i] = -1;
    queue<int> q;
    dist[s] = 0;
    q.push(s);
    while(!q.empty()){
        int u = q.front();
        q.pop();
        for(int i = 0; i < (int)e[u].size(); i++){
            int v = e[u][i];
            if(dist[v] == -1){
                dist[v] = dist[u] + 1;
                q.push(v);
            }
        }
    }
}

int farthest(int dist[]){
    int p = 1;
    for(int i = 2; i <= n; i++){
        if(dist[i] > dist[p]) p = i;
    }
    return p;
}

int main(){
    freopen("widegraph.in","r",stdin);
    freopen("widegraph.out","w",stdout);
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin >> n;
    for(int i = 1; i < n; i++){
        int u,v;
        cin >> u >> v;
        e[u].push_back(v);
        e[v].push_back(u);
    }
    bfs(1,dist1);
    int s = farthest(dist1);
    bfs(s,dists);
    int t = farthest(dists);
    bfs(t,distt);
    for(int i = 1; i <= n; i++){
        ecc[i] = max(dists[i],distt[i]);
    }
    sort(ecc + 1,ecc + n + 1);
    int diameter = ecc[n];
    for(int k = 1; k <= n; k++){
        if(k > diameter){
            cout << n;
        }else{
            int isolated = (int)(lower_bound(ecc + 1,ecc + n + 1,k) - ecc - 1);
            cout << 1 + isolated;
        }
        cout << (k == n ? '\n' : ' ');
    }
    return 0;
}
```

树上所有点的最远距离可以由一对直径端点概括。确定主连通块后，剩余顶点只要比较自己的偏心距与 k；排序一次即可回答全部阈值。